AIML Python Programs

Week-1

Finds

```
import pandas as pd

data = pd.read_csv("data.csv")

attributes = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

hypothesis = []

for i in range(len(attributes[0])):
    hypothesis.append("0")

for i in range(len(attributes)):
    if target[i] == "Yes" or target[i] == 1:
        for j in range(len(attributes[i])):
            if hypothesis[j] == "0":
                hypothesis[j] = attributes[i][j]
            elif hypothesis[j] != attributes[i][j]:
                hypothesis[j] = "?"

print("Final Hypothesis:")
print(hypothesis)
```

Week-2

Knn

```
import pandas as pd
import random

data = pd.read_csv("iris.csv", header=None)

data = data.values.tolist()
```

```
random.shuffle(data)

X = [row[0:4] for row in data]
y = [row[4] for row in data]

train_size = int(0.6 * len(X))

X_train = X[:train_size]
y_train = y[:train_size]

X_test = X[train_size:]
y_test = y[train_size:]

print("Training data:", len(X_train))
print("Testing data:", len(X_test))

def euclidean_distance(point1, point2):
    distance = 0

    for i in range(len(point1)):
        distance = distance + (float(point1[i]) - float(point2[i])) ** 2

    return distance ** 0.5

def knn_predict(X_train, y_train, test_point, k):
    distances = []

    for i in range(len(X_train)):
        distance = euclidean_distance(test_point, X_train[i])
        distances.append((distance, y_train[i]))

    distances.sort(key=lambda x: x[0])

    neighbors = distances[:k]

    class_count = {}

    for distance, label in neighbors:
        if label in class_count:
            class_count[label] += 1
        else:
            class_count[label] = 1

    predicted_class = None
    highest_count = 0

    for label in class_count:
        if class_count[label] > highest_count:
            highest_count = class_count[label]
```

```
            predicted_class = label

    return predicted_class

k = 5

predictions = []

for test_point in X_test:
    prediction = knn_predict(X_train, y_train, test_point, k)
    predictions.append(prediction)

correct = 0

for i in range(len(y_test)):
    if predictions[i] == y_test[i]:
        correct += 1

accuracy = (correct / len(y_test)) * 100

print("Correct Predictions:", correct)
print("Total Test Data:", len(y_test))
print("Accuracy:", accuracy, "%")
```

Week-3

Kmeans

```
import pandas as pd
import random

data = pd.read_csv("iris.csv", header=None)

data = data.values.tolist()

random.shuffle(data)

X = [row[0:4] for row in data]

train_size = int(0.6 * len(X))

X_train = X[:train_size]
X_test = X[train_size:]

print("Training data:", len(X_train))
print("Testing data:", len(X_test))
```

```
def euclidean_distance(point1, point2):
    distance = 0

    for i in range(len(point1)):
        distance = distance + (float(point1[i]) - float(point2[i])) ** 2

    return distance ** 0.5

def calculate_centroid(cluster):
    centroid = []

    for j in range(len(cluster[0])):
        total = 0

        for i in range(len(cluster)):
            total = total + float(cluster[i][j])

        centroid.append(total / len(cluster))

    return centroid

def kmeans(X, k):
    centroids = random.sample(X, k)

    for iteration in range(100):

        clusters = [[] for i in range(k)]

        for point in X:
            distances = []

            for centroid in centroids:
                distance = euclidean_distance(point, centroid)
                distances.append(distance)

            nearest = distances.index(min(distances))

            clusters[nearest].append(point)

        new_centroids = []

        for cluster in clusters:
            if len(cluster) > 0:
                new_centroids.append(calculate_centroid(cluster))
            else:
                new_centroids.append(random.choice(X))

        if new_centroids == centroids:
```

```
            break

        centroids = new_centroids

    return centroids

def predict_clusters(X, centroids):
    predictions = []

    for point in X:
        distances = []

        for centroid in centroids:
            distance = euclidean_distance(point, centroid)
            distances.append(distance)

        cluster = distances.index(min(distances))
        predictions.append(cluster)

    return predictions

k = 3

centroids = kmeans(X_train, k)

print("\nFinal Centroids:")

for i in range(len(centroids)):
    print("Cluster", i + 1, ":", centroids[i])

train_clusters = predict_clusters(X_train, centroids)

test_clusters = predict_clusters(X_test, centroids)

print("\nTraining Cluster Assignments:")
print(train_clusters)

print("\nTesting Cluster Assignments:")
print(test_clusters)

print("\nNumber of Testing Data Points in Each Cluster:")

for i in range(k):
    count = test_clusters.count(i)
    print("Cluster", i + 1, ":", count)
```

Week-4

Lwr

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

df = pd.read_csv("tips.csv")

x = df["total_bill"].values
y = df["tip"].values

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

tau = 5.0

x_train = np.column_stack([np.ones(len(x_train)), x_train])

predictions = []

for bill in x_test:
    weights = np.exp(-((x_train[:, 1] - bill) ** 2) / (2 * tau ** 2))
    W = np.diag(weights)
    xTW = np.dot(x_train.T, W)
    A = np.dot(xTW, x_train)
    b = np.dot(xTW, y_train)
    theta = np.dot(np.linalg.inv(A), b)
    prediction = theta[0] + theta[1] * bill
    predictions.append(prediction)

predictions = np.array(predictions)

sorted_indices = np.argsort(x_test)
x_test_sorted = x_test[sorted_indices]
predictions_sorted = predictions[sorted_indices]

plt.scatter(x_train[:, 1], y_train, color="lightblue", label="Training Data",
marker="o")
plt.scatter(x_test, y_test, color="darkblue", label="Test Data", marker="o")
plt.plot(x_test_sorted, predictions_sorted, color="red", label="Prediction
Line", linewidth=2)

plt.xlabel("Total Bill")
plt.ylabel("Tip")
plt.title("Locally Weighted Regression")
```

```
plt.legend()
plt.show()
```

Week-5

Enjoysport

```
import pandas as pd
import numpy as np

def calculate_entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    probabilities = counts / counts.sum()
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy

def calculate_information_gain(data, split_attribute_name, target_name):
    total_entropy = calculate_entropy(data[target_name])
    vals, counts = np.unique(data[split_attribute_name], return_counts=True)
    weighted_entropy = 0
    for i in range(len(vals)):
        subset = data[data[split_attribute_name] == vals[i]]
        subset_entropy = calculate_entropy(subset[target_name])
        weighted_entropy += (counts[i] / np.sum(counts)) * subset_entropy
    information_gain = total_entropy - weighted_entropy
    return information_gain

def build_decision_tree(data, original_data, features,
                        target_attribute_name, parent_node_class=None):
    if len(np.unique(data[target_attribute_name])) <= 1:
        return np.unique(data[target_attribute_name])[0]
    elif len(data) == 0:
        return np.unique(original_data[target_attribute_name])[
            np.argmax(np.unique(original_data[target_attribute_name],
                                return_counts=True)[1])]
    elif len(features) == 0:
        return parent_node_class
    else:
        np.unique(original_data[target_attribute_name])[
            np.argmax(np.unique(original_data[target_attribute_name],
                                return_counts=True)[1])]
        item_values = [
            calculate_information_gain(data, feature, target_attribute_name)
            for feature in features
        ]
        best_feature_index = np.argmax(item_values)
```

```
        best_feature = features[best_feature_index]
        tree = {best_feature: {}}
        features = [i for i in features if i != best_feature]

        for value in np.unique(data[best_feature]):
            subset_data = data.where(data[best_feature] == value).dropna()
            subtree = build_decision_tree(
                subset_data,
                original_data,
                features,
                target_attribute_name,
                parent_node_class
            )
            tree[best_feature][value] = subtree

        return tree

def print_tree(tree, indent=0):
    for key, value in tree.items():
        print(' ' * indent + str(key))
        if isinstance(value, dict):
            print_tree(value, indent + 1)
        else:
            print(' ' * (indent + 1) + 'Prediction: ' + str(value))

file_path = "/content/Day Ski AirTemp Humidity Wind Nate..."
df = pd.read_csv("enjoysport.csv")
features = df.columns[:-1].tolist()
target_col = df.columns[-1]
decision_tree = build_decision_tree(df, df, features, target_col)
print_tree(decision_tree)
```

Salaries

```
import pandas as pd
import numpy as np

def calculate_entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    probabilities = counts / np.sum(counts)
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy


def calculate_information_gain(data, split_attribute_name, target_name):
    total_entropy = calculate_entropy(data[target_name])
    vals, counts = np.unique(data[split_attribute_name], return_counts=True)
```

```
    weighted_entropy = 0
    for i in range(len(vals)):
        subset = data.where(data[split_attribute_name] == vals[i]).dropna()
        subset_entropy = calculate_entropy(subset[target_name])
        weighted_entropy += (counts[i] / np.sum(counts)) * subset_entropy
    information_gain = total_entropy - weighted_entropy
    return information_gain


def build_decision_tree(data, original_data, features,
                        target_attribute_name, parent_node_class=None):
    if len(np.unique(data[target_attribute_name])) <= 1:
        return np.unique(data[target_attribute_name])[0]
    elif len(data) == 0:
        return np.unique(original_data[target_attribute_name])
[np.argmax(np.unique(original_data[target_attribute_name], return_counts=True)
[1])]
    elif len(features) == 0:
        return parent_node_class
    else:
        np.unique(original_data[target_attribute_name])
[np.argmax(np.unique(data[target_attribute_name], return_counts=True)[1])]
        parent_node_class = np.unique(data[target_attribute_name],
return_counts=True)[1]

        item_values = [calculate_information_gain(data, feature,
                                                  target_attribute_name) for
feature in features]
        best_feature_index = np.argmax(item_values)
        best_feature = features[best_feature_index]

        tree = {best_feature: {}}
        features = [i for i in features if i != best_feature]
        for value in np.unique(data[best_feature]):
            sub_data = data.where(data[best_feature] == value).dropna()
            subtree = build_decision_tree(sub_data, original_data,
                                          features, target_attribute_name,
                                          parent_node_class)
            tree[best_feature][value] = subtree

        return tree


def print_tree(tree, indent=0):
    for key, value in tree.items():
        print(' ' * indent + str(key))
        if isinstance(value, dict):
            print_tree(value, indent + 1)
        else:
```

```
            print(' ' * (indent + 1) + 'Prediction: ' + str(value))

df = pd.read_csv("salaries.csv")
features = df.columns[:-1].tolist()
target_col = df.columns[-1]
decision_tree = build_decision_tree(df, df, features, target_col)
print_tree(decision_tree)
```

Titanic

```
import pandas as pd
import numpy as np

def calculate_entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    probabilities = counts / np.sum(counts)
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy

def calculate_information_gain(data, split_attribute_name, target_name):
    total_entropy = calculate_entropy(data[target_name])

    vals, counts = np.unique(data[split_attribute_name], return_counts=True)

    weighted_entropy = 0

    for i in range(len(vals)):
        subset = data[data[split_attribute_name] == vals[i]]
        subset_entropy = calculate_entropy(subset[target_name])
        weighted_entropy += (counts[i] / np.sum(counts)) * subset_entropy

    information_gain = total_entropy - weighted_entropy

    return information_gain

def build_decision_tree(data, original_data, features,
                        target_attribute_name, parent_node_class=None):

    if len(np.unique(data[target_attribute_name])) <= 1:
        return np.unique(data[target_attribute_name])[0]

    elif len(data) == 0:
        return np.unique(original_data[target_attribute_name])[
            np.argmax(
                np.unique(
                    original_data[target_attribute_name],
                    return_counts=True
                )[1]
```

```
            )
        ]

    elif len(features) == 0:
        return parent_node_class

    else:
        parent_node_class = np.unique(data[target_attribute_name])[
            np.argmax(
                np.unique(
                    data[target_attribute_name],
                    return_counts=True
                )[1]
            )
        ]

        item_values = [
            calculate_information_gain(
                data,
                feature,
                target_attribute_name
            )
            for feature in features
        ]

        best_feature_index = np.argmax(item_values)
        best_feature = features[best_feature_index]

        tree = {best_feature: {}}

        features = [
            i for i in features
            if i != best_feature
        ]

        for value in np.unique(data[best_feature]):
            sub_data = data[
                data[best_feature] == value
            ]

            subtree = build_decision_tree(
                sub_data,
                original_data,
                features,
                target_attribute_name,
                parent_node_class
            )

            tree[best_feature][value] = subtree
```

```
        return tree

def print_tree(tree, indent=0):
    for key, value in tree.items():
        print(" " * indent + str(key))

        if isinstance(value, dict):
            print_tree(value, indent + 1)
        else:
            print(
                " " * (indent + 1) +
                "Prediction: " +
                str(value)
            )

df = pd.read_csv("titanic.csv")

df = df[
    ["Survived", "Pclass", "Sex", "SibSp", "Parch", "Embarked"]
]

df = df.dropna()

df["Pclass"] = df["Pclass"].astype(str)
df["SibSp"] = df["SibSp"].astype(str)
df["Parch"] = df["Parch"].astype(str)

features = [
    "Pclass",
    "Sex",
    "SibSp",
    "Parch",
    "Embarked"
]

target_col = "Survived"

decision_tree = build_decision_tree(
    df,
    df,
    features,
    target_col
)

print_tree(decision_tree)
```

Decision Tree

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

data = pd.read_csv("titanic.csv")

X = data.drop("Survived", axis=1)
y = data["Survived"]

X = X.drop(["PassengerId", "Name", "Ticket", "Cabin"], axis=1)

X["Age"] = X["Age"].fillna(X["Age"].median())
X["Embarked"] = X["Embarked"].fillna(X["Embarked"].mode()[0])

X = pd.get_dummies(X, columns=["Sex", "Embarked"], dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predictions:", y_pred)
print("Actual:", y_test.values)
print("Accuracy:", accuracy_score(y_test, y_pred))

plt.figure(figsize=(15, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Not Survived", "Survived"],
    filled=True
)

plt.show()
```

Enjoy Sport Dt

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder

data = pd.read_csv("enjoysport.csv")

X = data.iloc[:, 1:-1]
y = data.iloc[:, -1]

encoders = {}

for column in X.columns:
    encoder = LabelEncoder()
    X[column] = encoder.fit_transform(X[column])
    encoders[column] = encoder

target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y)

model = DecisionTreeClassifier(criterion="entropy", random_state=42)

model.fit(X, y)

print("Accuracy:", model.score(X, y))

plt.figure(figsize=(12, 8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=target_encoder.classes_,
    filled=True
)

plt.show()
```

Salaries Dt

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder

data = pd.read_csv("salaries.csv")

X = data.iloc[:, :-1]
```

```
y = data.iloc[:, -1]

encoders = {}

for column in X.columns:
    encoder = LabelEncoder()
    X[column] = encoder.fit_transform(X[column])
    encoders[column] = encoder

model = DecisionTreeClassifier(criterion="entropy", random_state=42)

model.fit(X, y)

print("Accuracy:", model.score(X, y))

plt.figure(figsize=(15, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["0", "1"],
    filled=True
)

plt.show()
```

Titanic Dt

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

data = pd.read_csv("titanic.csv")

data = data.dropna()

data = data.drop(
    columns=["PassengerId", "Name", "Ticket", "Cabin"],
    errors="ignore"
)

X = data.drop("Survived", axis=1)
y = data["Survived"]

X = pd.get_dummies(X, dtype=int)

model = DecisionTreeClassifier(
    criterion="entropy",
```

```
    random_state=42
)

model.fit(X, y)

print("Accuracy:", model.score(X, y))

plt.figure(figsize=(20, 12))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Not Survived", "Survived"],
    filled=True
)

plt.show()
```

Week-6

Andgate

```
import pandas as pd
inputs = [
    (1, 1, 1),
    (1, 0, 0),
    (0, 1, 0),
    (0, 0, 0)
]
w1 = float(input("Enter Weight 1: "))
w2 = float(input("Enter Weight 2: "))
b = -1.5
L = 0.1
results = []
for x1, x2, gt in inputs:
    z = (x1 * w1) + (x2 * w2) + b
    if z >= 0:
        py = 1
    else:
        py = 0
    E = gt - py

    dw1 = L * E * x1
    dw2 = L * E * x2
    results.append([
        x1, x2, w1, w2, b, z, gt, py, E, L, dw1, dw2
```

```
    ])
columns = [
    "X1", "X2", "W1", "W2", "Bias",
    "Z", "GT", "PY", "E", "L",
    "ΔW1", "ΔW2"
]
df = pd.DataFrame(results, columns=columns)
print("\nAND GATE ANN")
print("=" * 100)
print(df.to_string(index=False))
print("=" * 100)
```

Orgate

```
import pandas as pd
data = [
    [1, 1, 1],
    [1, 0, 1],
    [0, 1, 1],
    [0, 0, 0]
]
w1 = float(input("Enter Weight 1: "))
w2 = float(input("Enter Weight 2: "))
b = -0.5
learning_rate = 0.1
rows = []
for x1, x2, gt in data:
    z = (x1 * w1) + (x2 * w2) + b
    py = 1 if z >= 0 else 0
    error = gt - py
    dw1 = learning_rate * error * x1
    dw2 = learning_rate * error * x2
    dw = abs(dw1) + abs(dw2)
    rows.append([
        x1, x2, gt, py, error,
        learning_rate, dw, w1, w2
    ])
table = pd.DataFrame(
    rows,
    columns=["X1", "X2", "GT", "PY", "E", "L", "ΔW", "W1", "W2"]
)
print("\nOR GATE - ANN")
print("=" * 70)
print(table.to_string(index=False))
print("=" * 70)
```

Mlp

```
import torch
x = torch.tensor([[1.0, 1.0]])
target = torch.tensor([[0.5]])
learning_rate = 1.0
epochs = 5
w1 = torch.tensor([
    [0.5, 0.5],
    [0.5, 0.5]
], requires_grad=True)
b1 = torch.tensor([[0.0, 0.0]], requires_grad=True)
w2 = torch.tensor([
    [0.5],
    [0.5]
], requires_grad=True)
b2 = torch.tensor([[0.0]], requires_grad=True)
results = []
for epoch in range(1, epochs + 1):
    hidden = torch.sigmoid(x @ w1 + b1)
    output = torch.sigmoid(hidden @ w2 + b2)
    loss = 0.5 * (target - output) ** 2
    loss.backward()
    with torch.no_grad():
        w1 -= learning_rate * w1.grad
        b1 -= learning_rate * b1.grad
        w2 -= learning_rate * w2.grad
        b2 -= learning_rate * b2.grad
    results.append([
        epoch,
        output.item(),
        target.item(),
        loss.item(),
        w1[0, 0].item(),
        w1[0, 1].item(),
        w1[1, 0].item(),
        w1[1, 1].item(),
        w2[0, 0].item(),
        w2[1, 0].item()
    ])
    w1.grad.zero_()
    b1.grad.zero_()
    w2.grad.zero_()
    b2.grad.zero_()
print("\nMLP USING BACKPROPAGATION")
print("Architecture: 2-2-1")
print("=" * 125)
print(
    f"{'Epoch':^7}"
    f"{'Actual Output':^16}"
    f"{'Target':^12}"
```

```
    f"{'Loss':^14}"
    f"{'W1(1,1)':^12}"
    f"{'W1(1,2)':^12}"
    f"{'W1(2,1)':^12}"
    f"{'W1(2,2)':^12}"
    f"{'W2(1)':^12}"
    f"{'W2(2)':^12}"
)
print("-" * 125)
for row in results:
    print(
        f"{row[0]:^7}"
        f"{row[1]:^16.6f}"
        f"{row[2]:^12.2f}"
        f"{row[3]:^14.6f}"
        f"{row[4]:^12.6f}"
        f"{row[5]:^12.6f}"
        f"{row[6]:^12.6f}"
        f"{row[7]:^12.6f}"
        f"{row[8]:^12.6f}"
        f"{row[9]:^12.6f}"
    )
print("=" * 125)
```

Week-7

Em

```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans
import pandas as pd
import numpy as np
from sklearn import preprocessing
from sklearn.mixture import GaussianMixture
iris = datasets.load_iris()

X = pd.DataFrame(iris.data)
X.columns = ['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width']

y = pd.DataFrame(iris.target)
y.columns = ['Targets']

model = KMeans(n_clusters=3, random_state=42)
model.fit(X)
```

```
plt.figure(figsize=(14, 14))

colormap = np.array(['red', 'lime', 'black'])

plt.subplot(2, 2, 1)
plt.scatter(
    X.Petal_Length,
    X.Petal_Width,
    c=colormap[y.Targets],
    s=40
)
plt.title('Real Clusters')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')

plt.subplot(2, 2, 2)
plt.scatter(
    X.Petal_Length,
    X.Petal_Width,
    c=colormap[model.labels_],
    s=40
)
plt.title('K-Means Clustering')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')

scaler = preprocessing.StandardScaler()
scaler.fit(X)

xsa = scaler.transform(X)
xs = pd.DataFrame(xsa, columns=X.columns)


gmm = GaussianMixture(n_components=3, random_state=42)
gmm.fit(xs)

gmm_y = gmm.predict(xs)

plt.subplot(2, 2, 3)
plt.scatter(
    X.Petal_Length,
    X.Petal_Width,
    c=colormap[gmm_y],
    s=40
)
plt.title('GMM Clustering')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
```

```
plt.tight_layout()
plt.show()

print('Observation: The GMM using EM algorithm based clustering matched the
true labels more closely than the Kmeans.')
```

Week-8

Bfs

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = 'A'
goal = 'G'

open_list = deque([start])
closed_list = []
parent = {start: None}

while open_list:
    node = open_list.popleft()

    if node in closed_list:
        continue

    closed_list.append(node)

    print("OPEN:", list(open_list))
    print("CLOSED:", closed_list)

    if node == goal:
        break

    for neighbour in graph[node]:
        if neighbour not in closed_list and neighbour not in open_list:
            parent[neighbour] = node
```

```
            open_list.append(neighbour)

path = []

if goal in closed_list:
    node = goal

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()

print("\nPath:", " -> ".join(path))
print("Final OPEN:", list(open_list))
print("Final CLOSED:", closed_list)
```

Dfs

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = 'A'
goal = 'G'

open_list = [start]
closed_list = []
parent = {start: None}

while open_list:
    node = open_list.pop()

    if node in closed_list:
        continue

    closed_list.append(node)

    print("OPEN:", open_list)
    print("CLOSED:", closed_list)

    if node == goal:
```

```
        break

    for neighbour in reversed(graph[node]):
        if neighbour not in closed_list and neighbour not in open_list:
            parent[neighbour] = node
            open_list.append(neighbour)

path = []

if goal in closed_list:
    node = goal

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()

print("\nPath:", " -> ".join(path))
print("Final OPEN:", open_list)
print("Final CLOSED:", closed_list)
```

Iddfs

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = 'A'
goal = 'G'


def depth_limited_search(start, goal, limit):

    open_list = [(start, 0)]
    closed_list = []
    parent = {start: None}

    while open_list:

        node, depth = open_list.pop()
```

```
        if node in closed_list:
            continue

        closed_list.append(node)

        print("OPEN:", open_list)
        print("CLOSED:", closed_list)

        if node == goal:

            path = []
            current = node

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()

            return path, open_list, closed_list

        if depth < limit:

            for neighbour in reversed(graph[node]):

                if neighbour not in closed_list:
                    parent[neighbour] = node
                    open_list.append((neighbour, depth + 1))

    return None, open_list, closed_list


depth = 0

while True:

    print("\n==============================")
    print("DEPTH LIMIT:", depth)
    print("==============================")

    path, open_list, closed_list = depth_limited_search(
        start,
        goal,
        depth
    )

    if path is not None:
        print("\nGoal Found!")
        print("Path:", " -> ".join(path))
```

```
        print("Final OPEN:", open_list)
        print("Final CLOSED:", closed_list)
        break

    depth += 1
```